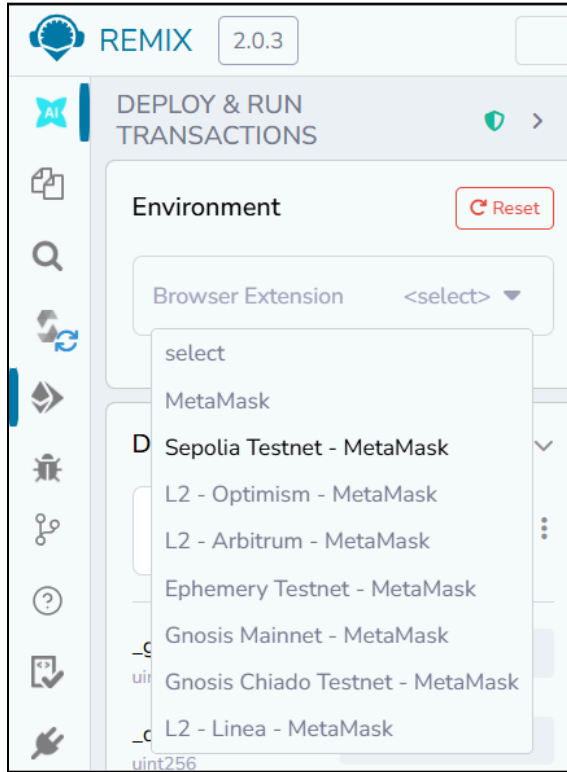


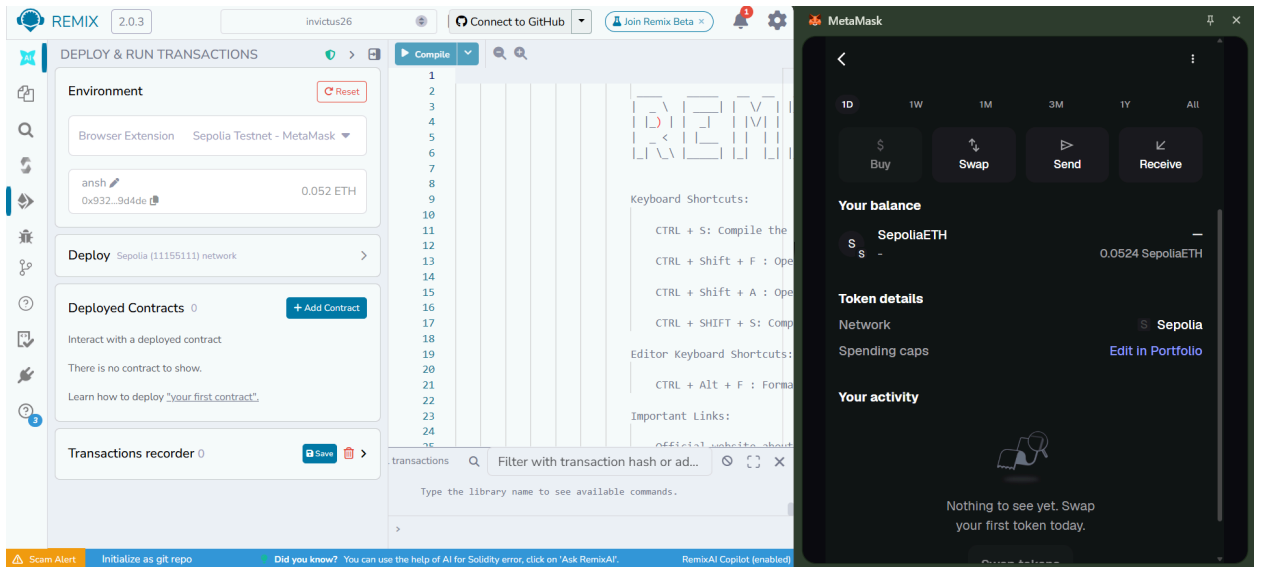
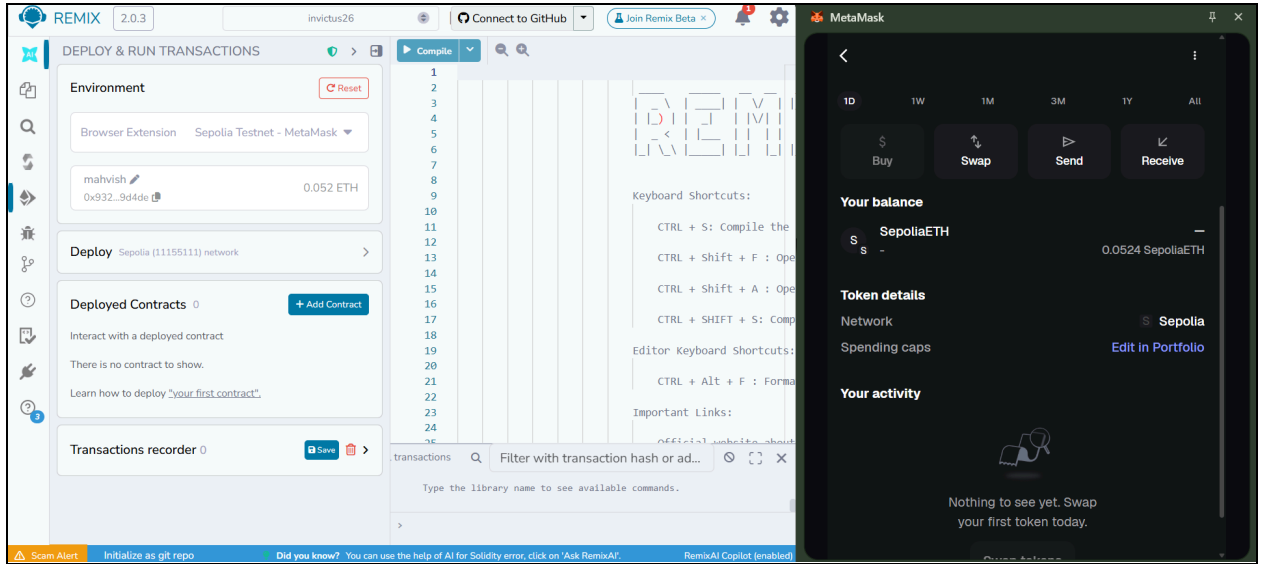
Experiment 7

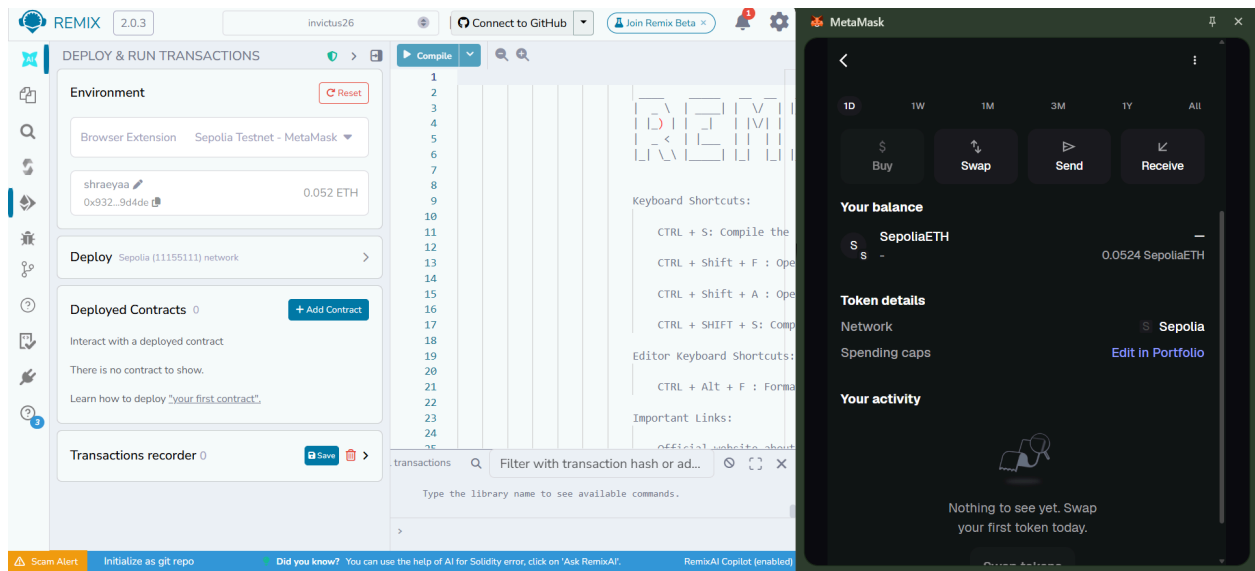
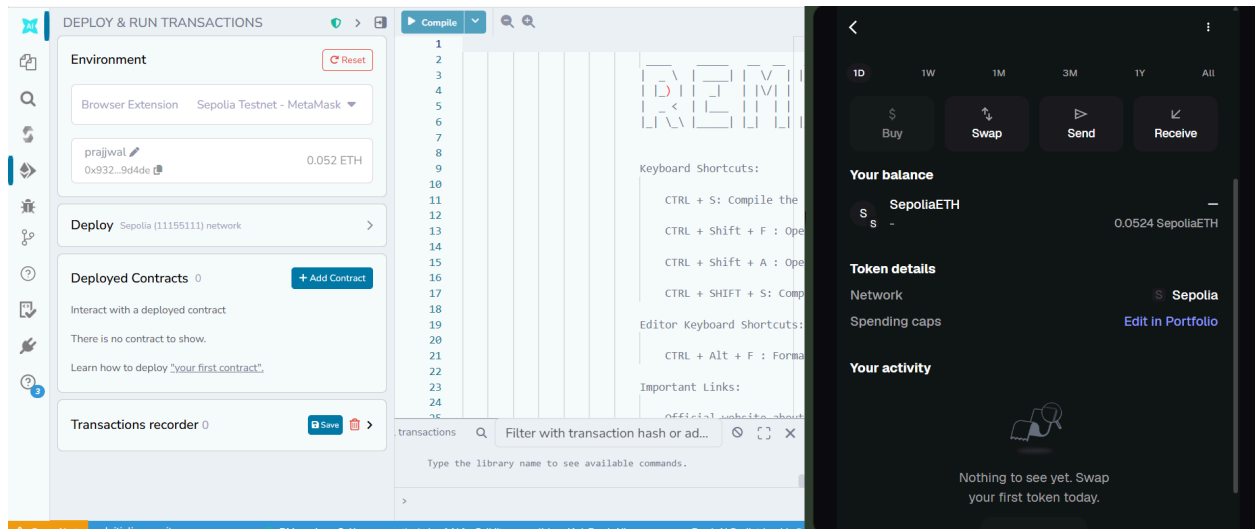
Aim: Implement the embedding wallet (Metamask) and transaction using Solidity

Step 1: Select metamask sepolia testnet in environment



Initial setup:





Step 2: Create a contract for a crowdfunding platform

Code:

```
// SPDX-License-Identifier: MIT
```

```
pragma solidity ^0.8.0;
```

```
contract Crowdfunding {
    address public owner;
    uint public goal;
    uint public deadline;
    uint public totalFunds;
```

```

mapping(address => uint) public contributions;

constructor(uint _goal, uint _duration) {
    owner = msg.sender;
    goal = _goal;
    deadline = block.timestamp + _duration;
}

function contribute() public payable {
    require(block.timestamp < deadline, "Deadline passed");
    require(msg.value > 0, "Send some ETH");

    contributions[msg.sender] += msg.value;
    totalFunds += msg.value;
}

function withdrawFunds() public {
    require(msg.sender == owner, "Only owner");
    require(totalFunds >= goal, "Goal not reached");

    (bool success, ) = payable(owner).call{value: totalFunds}("");
    require(success, "Transfer failed");
}

function refund() public {
    require(block.timestamp > deadline, "Not ended");
    require(totalFunds < goal, "Goal was reached");

    uint amount = contributions[msg.sender];
    require(amount > 0, "No contribution");

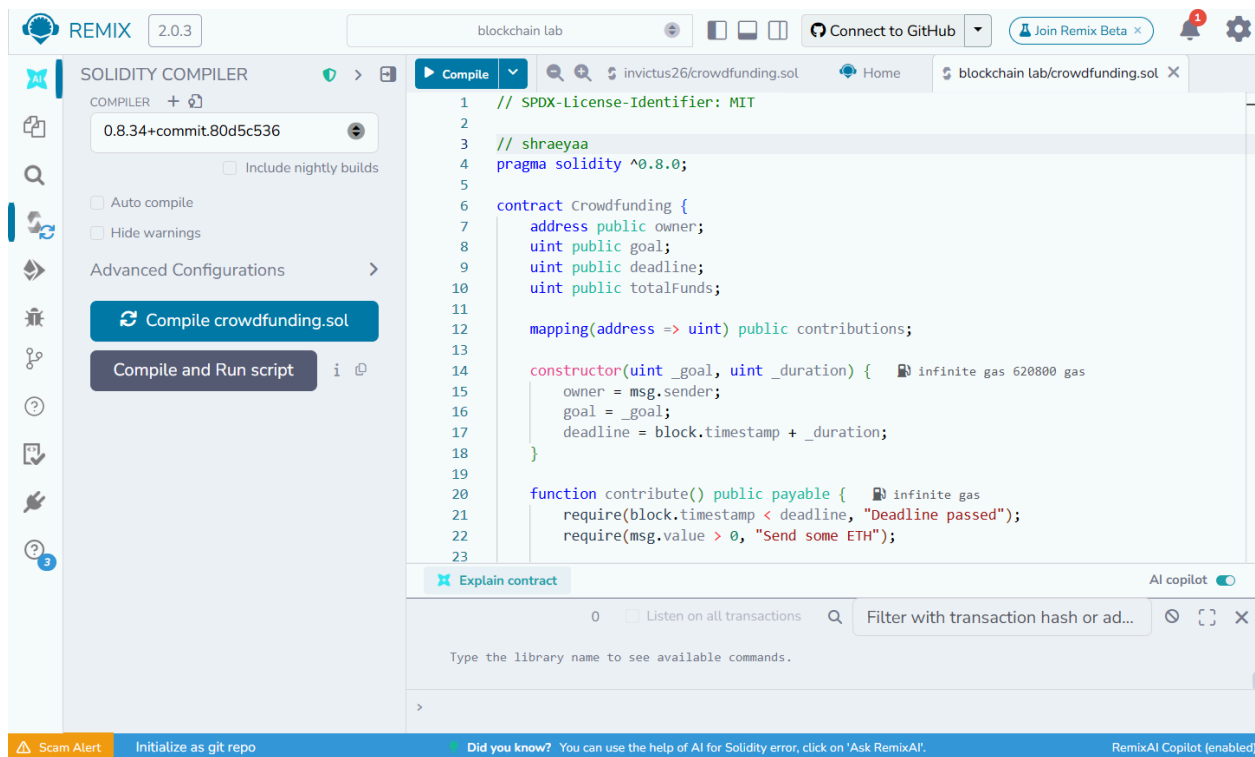
    contributions[msg.sender] = 0;

    (bool success, ) = payable(msg.sender).call{value: amount}("");
    require(success, "Refund failed");
}

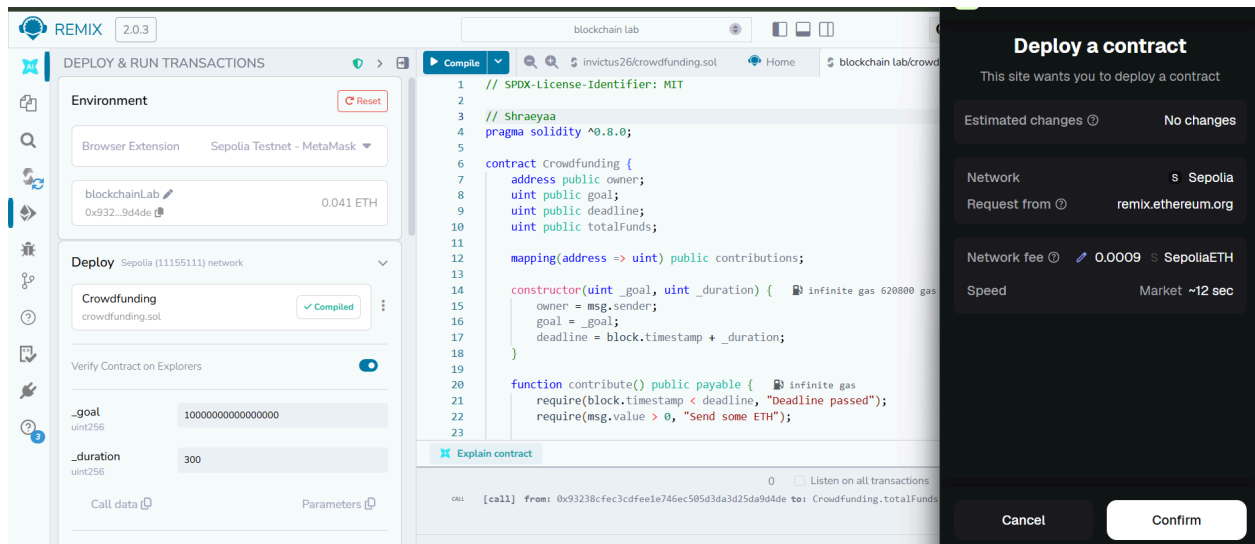
function getBalance() public view returns (uint) {
    return address(this).balance;
}
}

```

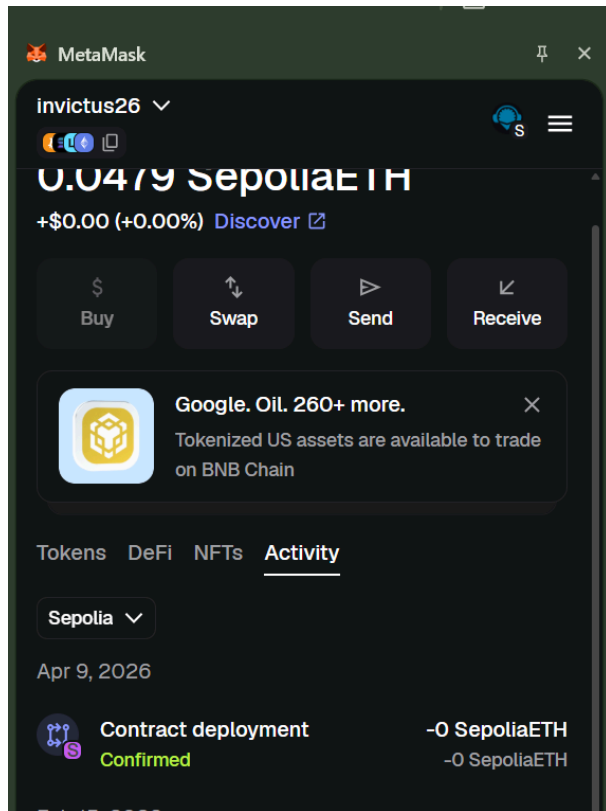
Step 3: save it as **crowdfunding.sol** and compile it



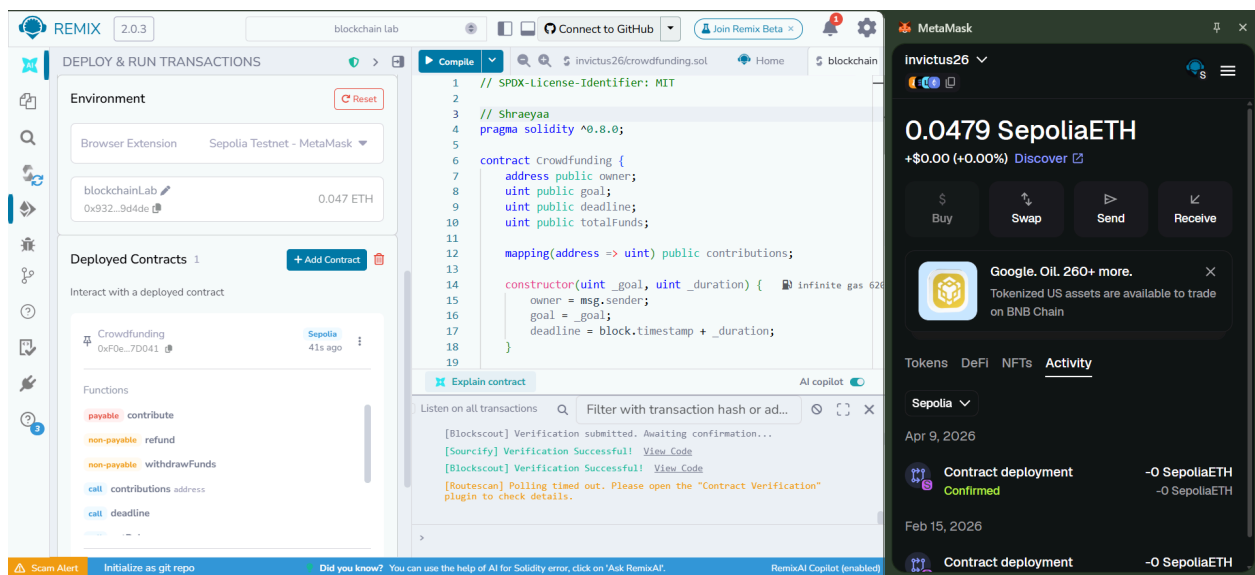
Step 4: deploy the contract with an initial goal and time limit. Confirm the transaction from metamask



After deployment you can see: transaction recorded in metamask



Step 5: After deployment in remix ide, we can see the deployed contract and its functions



Step 6: Add a contribution of 0.01 ETH = 10000000000000000 wei. Transaction request appears in metamask. Click on confirm

Step 7: copy paste the contracts address in <https://sepolia.etherscan.io/> to see all transactions:

The screenshot shows the Etherscan.io interface for a contract on the Sepolia Testnet. The contract address is 0xc160840C08F938f53951A204ebfC0EfA2B6b8BFe. The page is divided into three main sections: Overview, More Info, and Multichain Info. The Overview section shows an ETH balance of 0.001 ETH. The More Info section shows the contract creator as 0x93238CfE...25dA9D4DE, who created it 9 minutes ago. The Multichain Info section shows N/A. Below these sections, there are tabs for Transactions, Token Transfers (ERC-20), Contract, and Events. The Transactions tab is selected, showing a list of transactions. The first transaction is a 'Contribute' transaction with a hash of 0x8afc9747c4d... and a value of 0.001 ETH. The transaction was made 48 seconds ago from the contract address to the same address.

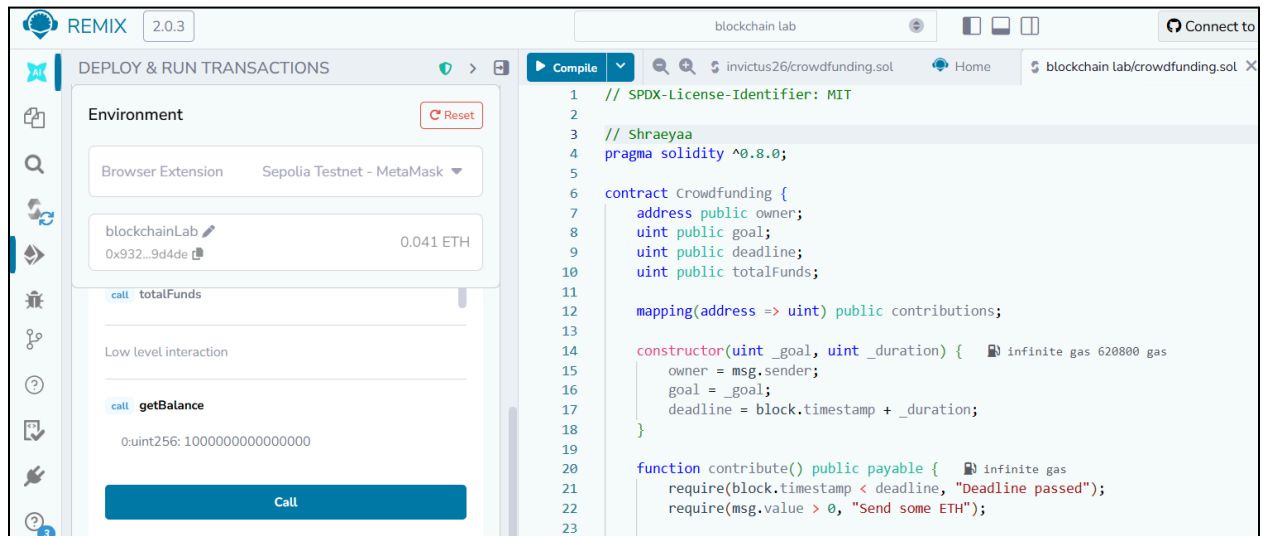
Transaction Hash	Method	Block	Age	From	To	Amount
0x8afc9747c4d...	Contribute	10623857	48 secs ago	0x93238CfE...25dA9D4DE	0xc160840C...A2B6b8BFe	0.001 ETH

Also can see the transactions under all transactions in remix IDE

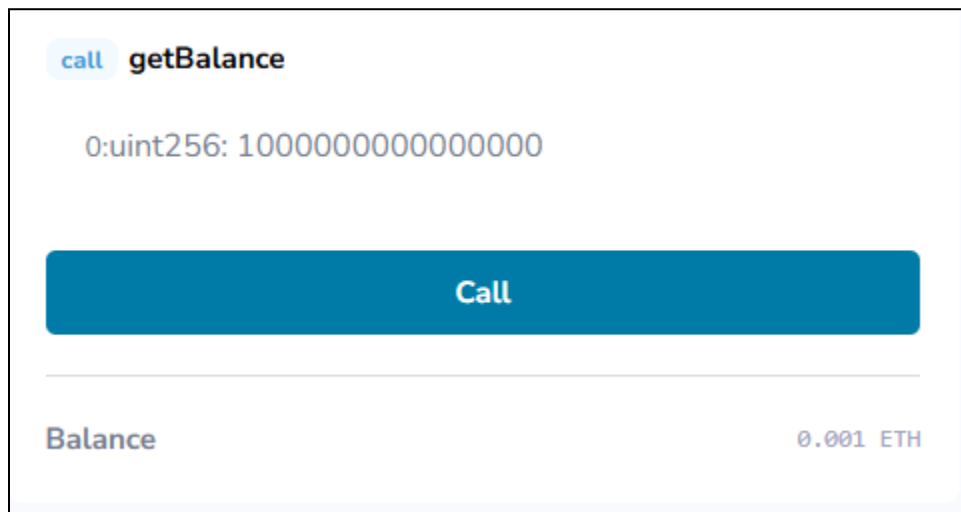
The screenshot shows the 'Transactions recorder' in the Remix IDE. It has a 'Save' button and a dropdown menu. There are two tabs: 'Contract call' and 'Transaction list', with 'Transaction list' selected. The list shows two transactions. The first is a 'contribute' transaction with a status of 'payable' and a value of 5min ago. The second is a 'Crowdfunding' transaction with a status of 'deploy' and a value of 13min ago. Both transactions have a green checkmark indicating they were successful.

Transaction	Status	Value
contribute tx: 0x8af...39de9	payable	5min ago
Crowdfunding tx: 0x124...548ed	deploy	13min ago

Step 8: use getBalance function to check the total funds received



Output:



Step 9: Paste wallet address from metamask into contributions to see all transactions by the particular use

call

contributions

address

0x93238CfEc3cdfEE1E746eC505d3DA3D25dA9D4DE

0:uint256: 10000000000000000

Call

Step 10: use totalFunds function to see totalFunds recieved

call

totalFunds

0:uint256: 10000000000000000

Call